



Framework

AWS Well-Architected Framework



AWS Well-Architected Framework: Framework

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Abstract and introduction	1
Introduction	1
Definitions	2
On architecture	4
General design principles	6
The pillars of the framework	8
Operational excellence	8
Design principles	9
Definition	10
Best practices	11
Resources	20
Security	20
Design principles	21
Definition	21
Best practices	22
Resources	31
Reliability	32
Design principles	32
Definition	33
Best practices	34
Resources	39
Performance efficiency	39
Design principles	40
Definition	40
Best practices	41
Resources	46
Cost optimization	46
Design principles	47
Definition	48
Best practices	48
Resources	54
Sustainability	55
Design principles	55
Definition	56

Best practices	57
Resources	63
The review process	64
Conclusion	66
Contributors	67
Further reading	68
Document revisions	69
Appendix: Questions and best practices	73
Operational excellence	73
Organization	73
Prepare	130
Operate	198
Evolve	241
Security	260
Security foundations	261
Identity and access management	285
Detection	343
Infrastructure protection	357
Data protection	383
Incident response	417
Application security	440
Reliability	464
Foundations	464
Workload architecture	504
Change management	553
Failure management	600
Performance efficiency	701
Architecture selection	702
Compute and hardware	716
Data management	734
Networking and content delivery	757
Process and culture	786
Cost optimization	803
Practice Cloud Financial Management	803
Expenditure and usage awareness	826
Cost-effective resources	867

Manage demand and supply resources	908
Optimize over time	920
Sustainability	928
Region selection	929
Alignment to demand	931
Software and architecture	945
Data	957
Hardware and services	975
Process and culture	985
Notices	996
AWS Glossary	997

AWS Well-Architected Framework

Publication date: **November 6, 2024** ([Document revisions](#))

The AWS Well-Architected Framework helps you understand the pros and cons of decisions you make while building systems on AWS. By using the Framework you will learn architectural best practices for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems in the cloud.

Introduction

The AWS Well-Architected Framework helps you understand the pros and cons of decisions you make while building systems on AWS. Using the Framework helps you learn architectural best practices for designing and operating secure, reliable, efficient, cost-effective, and sustainable workloads in the AWS Cloud. It provides a way for you to consistently measure your architectures against best practices and identify areas for improvement. The process for reviewing an architecture is a constructive conversation about architectural decisions, and is not an audit mechanism. We believe that having well-architected systems greatly increases the likelihood of business success.

AWS Solutions Architects have years of experience architecting solutions across a wide variety of business verticals and use cases. We have helped design and review thousands of customers' architectures on AWS. From this experience, we have identified best practices and core strategies for architecting systems in the cloud.

The AWS Well-Architected Framework documents a set of foundational questions that help you to understand if a specific architecture aligns well with cloud best practices. The framework provides a consistent approach to evaluating systems against the qualities you expect from modern cloud-based systems, and the remediation that would be required to achieve those qualities. As AWS continues to evolve, and we continue to learn more from working with our customers, we will continue to refine the definition of well-architected.

This framework is intended for those in technology roles, such as chief technology officers (CTOs), architects, developers, and operations team members. It describes AWS best practices and strategies to use when designing and operating a cloud workload, and provides links to further implementation details and architectural patterns. For more information, see the [AWS Well-Architected homepage](#).

AWS also provides a service for reviewing your workloads at no charge. The [AWS Well-Architected Tool](#) (AWS WA Tool) is a service in the cloud that provides a consistent process for you to review and measure your architecture using the AWS Well-Architected Framework. The AWS WA Tool provides recommendations for making your workloads more reliable, secure, efficient, and cost-effective.

To help you apply best practices, we have created [AWS Well-Architected Labs](#), which provides you with a repository of code and documentation to give you hands-on experience implementing best practices. We also have teamed up with select AWS Partner Network (APN) Partners, who are members of the [AWS Well-Architected Partner program](#). These AWS Partners have deep AWS knowledge, and can help you review and improve your workloads.

Definitions

Every day, experts at AWS assist customers in architecting systems to take advantage of best practices in the cloud. We work with you on making architectural trade-offs as your designs evolve. As you deploy these systems into live environments, we learn how well these systems perform and the consequences of those trade-offs.

Based on what we have learned, we have created the AWS Well-Architected Framework, which provides a consistent set of best practices for customers and partners to evaluate architectures, and provides a set of questions you can use to evaluate how well an architecture is aligned to AWS best practices.

The AWS Well-Architected Framework is based on six pillars — operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability.

Table 1. The pillars of the AWS Well-Architected Framework

Name	Description
Operational excellence	The ability to support development and run workloads effectively, gain insight into their operations, and to continuously improve supporting processes and procedures to deliver business value.
Security	The security pillar describes how to take advantage of cloud technologies to protect

Name	Description
	data, systems, and assets in a way that can improve your security posture.
Reliability	The reliability pillar encompasses the ability of a workload to perform its intended function correctly and consistently when it's expected to. This includes the ability to operate and test the workload through its total lifecycle . This paper provides in-depth, best practice guidance for implementing reliable workloads on AWS.
Performance efficiency	The ability to use computing resources efficiently to meet system requirements, and to maintain that efficiency as demand changes and technologies evolve.
Cost optimization	The ability to run systems to deliver business value at the lowest price point.
Sustainability	The ability to continually improve sustainability impacts by reducing energy consumption and increasing efficiency across all components of a workload by maximizing the benefits from the provisioned resources and minimizing the total resources required.

In the AWS Well-Architected Framework, we use these terms:

- A **component** is the code, configuration, and AWS Resources that together deliver against a requirement. A component is often the unit of technical ownership, and is decoupled from other components.
- The term **workload** is used to identify a set of components that together deliver business value. A workload is usually the level of detail that business and technology leaders communicate about.

- We think about **architecture** as being how components work together in a workload. How components communicate and interact is often the focus of architecture diagrams.
- **Milestones** mark key changes in your architecture as it evolves throughout the product lifecycle (design, implementation, testing, go live, and in production).
- Within an organization the **technology portfolio** is the collection of workloads that are required for the business to operate.
- The **level of effort** is categorizing the amount of time, effort, and complexity a task requires for implementation. Each organization needs to consider the size and expertise of the team and the complexity of the workload for additional context to properly categorize the level of effort for the organization.
 - **High:** The work might take multiple weeks or multiple months. This could be broken out into multiple stories, releases, and tasks.
 - **Medium:** The work might take multiple days or multiple weeks. This could be broken out into multiple releases and tasks.
 - **Low:** The work might take multiple hours or multiple days. This could be broken out into multiple tasks.

When architecting workloads, you make trade-offs between pillars based on your business context. These business decisions can drive your engineering priorities. You might optimize to improve sustainability impact and reduce cost at the expense of reliability in development environments, or, for mission-critical solutions, you might optimize reliability with increased costs and sustainability impact. In ecommerce solutions, performance can affect revenue and customer propensity to buy. Security and operational excellence are generally not traded-off against the other pillars.

On architecture

In on-premises environments, customers often have a central team for technology architecture that acts as an overlay to other product or feature teams to verify they are following best practice. Technology architecture teams typically include a set of roles such as: Technical Architect (infrastructure), Solutions Architect (software), Data Architect, Networking Architect, and Security Architect. Often these teams use [TOGAF](#) or the [Zachman Framework](#) as part of an enterprise architecture capability.

At AWS, we prefer to distribute capabilities into teams rather than having a centralized team with that capability. There are risks when you choose to distribute decision making authority, for

example, verifying that teams are meeting internal standards. We mitigate these risks in two ways. First, we have *practices* (ways of doing things, process, standards, and accepted norms) that focus on allowing each team to have that capability, and we put in place experts who verify that teams raise the bar on the standards they need to meet. Second, we implement *mechanisms* that carry out automated checks to verify standards are being met.

 “Good intentions never work, you need good mechanisms to make anything happen” — Jeff Bezos.

This means replacing a human's best efforts with mechanisms (often automated) that check for compliance with rules or process. This distributed approach is supported by the [Amazon leadership principles](#), and establishes a culture across all roles that *works back* from the customer. Working backward is a fundamental part of our innovation process. We start with the customer and what they want, and let that define and guide our efforts. Customer-obsessed teams build products in response to a customer need.

For architecture, this means that we expect every team to have the capability to create architectures and to follow best practices. To help new teams gain these capabilities or existing teams to raise their bar, we activate access to a virtual community of principal engineers who can review their designs and help them understand what AWS best practices are. The principal engineering community works to make best practices visible and accessible. One way they do this, for example, is through lunchtime talks that focus on applying best practices to real examples. These talks are recorded and can be used as part of onboarding materials for new team members.

AWS best practices emerge from our experience running thousands of systems at internet scale. We prefer to use data to define best practice, but we also use subject matter experts, like principal engineers, to set them. As principal engineers see new best practices emerge, they work as a community to verify that teams follow them. In time, these best practices are formalized into our internal review processes, and also into mechanisms that enforce compliance. The Well-Architected Framework is the customer-facing implementation of our internal review process, where we have codified our principal engineering thinking across field roles, like Solutions Architecture and internal engineering teams. The Well-Architected Framework is a scalable mechanism that lets you take advantage of these learnings.

By following the approach of a principal engineering community with distributed ownership of architecture, we believe that a Well-Architected enterprise architecture can emerge that is driven

by customer need. Technology leaders (such as a CTOs or development managers), carrying out Well-Architected reviews across all your workloads will permit you to better understand the risks in your technology portfolio. Using this approach, you can identify themes across teams that your organization could address by mechanisms, training, or lunchtime talks where your principal engineers can share their thinking on specific areas with multiple teams.

General design principles

The Well-Architected Framework identifies a set of general design principles to facilitate good design in the cloud:

- **Stop guessing your capacity needs:** If you make a poor capacity decision when deploying a workload, you might end up sitting on expensive idle resources or dealing with the performance implications of limited capacity. With cloud computing, these problems can go away. You can use as much or as little capacity as you need, and scale in and out automatically.
- **Test systems at production scale:** In the cloud, you can create a production-scale test environment on demand, complete your testing, and then decommission the resources. Because you only pay for the test environment when it's running, you can simulate your live environment for a fraction of the cost of testing on premises.
- **Automate with architectural experimentation in mind:** Automation permits you to create and replicate your workloads at low cost and avoid the expense of manual effort. You can track changes to your automation, audit the impact, and revert to previous parameters when necessary.
- **Consider evolutionary architectures:** In a traditional environment, architectural decisions are often implemented as static, onetime events, with a few major versions of a system during its lifetime. As a business and its context continue to evolve, these initial decisions might hinder the system's ability to deliver changing business requirements. In the cloud, the capability to automate and test on demand lowers the risk of impact from design changes. This permits systems to evolve over time so that businesses can take advantage of innovations as a standard practice.
- **Drive architectures using data:** In the cloud, you can collect data on how your architectural choices affect the behavior of your workload. This lets you make fact-based decisions on how to improve your workload. Your cloud infrastructure is code, so you can use that data to inform your architecture choices and improvements over time.
- **Improve through game days:** Test how your architecture and processes perform by regularly scheduling game days to simulate events in production. This will help you understand where

improvements can be made and can help develop organizational experience in dealing with events.

The pillars of the framework

Creating a software system is a lot like constructing a building. If the foundation is not solid, structural problems can undermine the integrity and function of the building. When architecting technology solutions, if you neglect the six pillars of operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability, it can become challenging to build a system that delivers on your expectations and requirements. Incorporating these pillars into your architecture will help you produce stable and efficient systems. This will allow you to focus on the other aspects of design, such as functional requirements.

Pillars

- [Operational excellence](#)
- [Security](#)
- [Reliability](#)
- [Performance efficiency](#)
- [Cost optimization](#)
- [Sustainability](#)

Operational excellence

Operational excellence (OE) is a commitment to build software correctly while consistently delivering a great customer experience. The operational excellence pillar contains best practices for organizing your team, designing your workload, operating it at scale, and evolving it over time.

The operational excellence pillar provides an overview of design principles, best practices, and questions. You can find prescriptive guidance on implementation in the [Operational Excellence Pillar whitepaper](#).

Topics

- [Design principles](#)
- [Definition](#)
- [Best practices](#)
- [Resources](#)

Design principles

The following are design principles for operational excellence in the cloud:

- **Organize teams around business outcomes:** The ability of a team to achieve business outcomes comes from leadership vision, effective operations, and a business-aligned operating model. Leadership should be fully invested and committed to a CloudOps transformation with a suitable cloud operating model that incentivizes teams to operate in the most efficient way and meet business outcomes. The right operating model uses people, process, and technology capabilities to scale, optimize for productivity, and differentiate through agility, responsiveness, and adaptation. The organization's long-term vision is translated into goals that are communicated across the enterprise to stakeholders and consumers of your cloud services. Goals and operational KPIs are aligned at all levels. This practice sustains the long-term value derived from implementing the following design principles.
- **Implement observability for actionable insights:** Gain a comprehensive understanding of workload behavior, performance, reliability, cost, and health. Establish key performance indicators (KPIs) and leverage observability telemetry to make informed decisions and take prompt action when business outcomes are at risk. Proactively improve performance, reliability, and cost based on actionable observability data.
- **Safely automate where possible:** In the cloud, you can apply the same engineering discipline that you use for application code to your entire environment. You can define your entire workload and its operations (applications, infrastructure, configuration, and procedures) as code, and update it. You can then automate your workload's operations by initiating them in response to events. In the cloud, you can employ automation safety by configuring guardrails, including rate control, error thresholds, and approvals. Through effective automation, you can achieve consistent responses to events, limit human error, and reduce operator toil.
- **Make frequent, small, reversible changes:** Design workloads that are scalable and loosely coupled to permit components to be updated regularly. Automated deployment techniques together with smaller, incremental changes reduces the blast radius and allows for faster reversal when failures occur. This increases confidence to deliver beneficial changes to your workload while maintaining quality and adapting quickly to changes in market conditions.
- **Refine operations procedures frequently:** As you evolve your workloads, evolve your operations appropriately. As you use operations procedures, look for opportunities to improve them. Hold regular reviews and validate that all procedures are effective and that teams are familiar with them. Where gaps are identified, update procedures accordingly. Communicate procedural

updates to all stakeholders and teams. Gamify your operations to share best practices and educate teams.

- **Anticipate failure:** Maximize operational success by driving failure scenarios to understand the workload's risk profile and its impact on your business outcomes. Test the effectiveness of your procedures and your team's response against these simulated failures. Make informed decisions to manage open risks that are identified by your testing.
- **Learn from all operational events and metrics:** Drive improvement through lessons learned from all operational events and failures. Share what is learned across teams and through the entire organization. Learnings should highlight data and anecdotes on how operations contribute to business outcomes.
- **Use managed services:** Reduce operational burden by using AWS managed services where possible. Build operational procedures around interactions with those services.

Definition

There are four best practice areas for operational excellence in the cloud:

- **Organization**
- **Prepare**
- **Operate**
- **Evolve**

Your organization's leadership defines business objectives. Your organization must understand requirements and priorities and use these to organize and conduct work to support the achievement of business outcomes. Your workload must emit the information necessary to support it. Implementing services to achieve integration, deployment, and delivery of your workload will create an increased flow of beneficial changes into production by automating repetitive processes.

There may be risks inherent in the operation of your workload. Understand those risks and make an informed decision to enter production. Your teams must be able to support your workload. Business and operational metrics derived from desired business outcomes will permit you to understand the health of your workload, your operations activities, and respond to incidents. Your priorities will change as your business needs and business environment changes. Use these as a feedback loop to continually drive improvement for your organization and the operation of your workload.